

FUSEMACHINE PROJECT

**ResearchX: An Autonomous AI Agent Harness
for Research and Paper Writing**

A Literature Review and Implementation Audit

PREPARED BY

Chandan Shah

Viraj Sawad

Janvi Rajbhandari

PREPARED FOR THE FUSEMACHINE PROJECT

SEPTEMBER, 2026

ABSTRACT

Research does not fail for lack of papers alone. It fails when discovery, execution, verification and writing are disconnected. This review examines ResearchX as a proposed terminal (TUI) research agent: a question enters, the system assembles source-backed evidence and experiment artifacts, and a human must approve any final draft. Chapter 2 states the problem and six acceptance criteria. Chapter 3 reviews the Pi harness, related research agents and research-writing products, and the open discovery sources. Chapter 4 records the previously discussed architecture and the design corrections. Chapter 5 constructs a comparative capability pipeline. Chapter 6 documents the source stack and the current verification limitation. Chapter 7 gives the proposed ResearchX design and audits its implementation status.

The audit is deliberately conservative. Pi-native integration, PaperRank, science-database search, Kaggle tools, the skeptic workflow and the persisted human checkpoint are present in the checked-out source tree. The citation verifier still calls Semantic Scholar, and a dedicated hypothesis-branch, append-only findings-ledger and doom-loop implementation is not yet evidenced as a completed ResearchX feature. Those gaps are recorded instead of being presented as shipped capabilities.

Contents

ABSTRACT	i
TABLE OF CONTENTS	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
ABBREVIATIONS	vi
1 INTRODUCTION	1
2 PROBLEM STATEMENT	3
2.1 Context	3
2.2 Problem Formulation	3
2.3 Acceptance Criteria	3
3 LITERATURE REVIEW	5
3.1 Review of Related Theory	5
3.1.1 Agent Harness and Tool Extension Model	5
3.1.2 Paper Ranking with Citation Graphs	5
3.1.3 Citation Verification	5
3.1.4 Stage-Gate Integrity	6
3.2 Review of Related Works	6
3.2.1 Feynman-Class Research Agents	6
3.2.2 Execution and Guarding	6
3.2.3 Skills Methodology	7
3.2.4 Local-First Research Studios	7
3.2.5 Discovery Sources	7
3.2.6 Paid Research Platforms	7
3.3 Research Gap	8
4 PREVIOUSLY USED ARCHITECTURE	9
4.1 The Discussed Design	9
4.2 What Changed and Why	9
5 COMPARATIVE ARCHITECTURE	11
5.1 A Composite Capability Pipeline	11
5.2 What the Composite Does Not Prove	11

6	SOURCE STACK AND VERIFICATION LIMITS	12
7	PROPOSED SOLUTION	13
7.1	Design on the Pi Base	13
7.2	Why This Approach Is a Coherent Integration	13
8	CONCLUSION AND NEXT WORK	15

List of Figures

1.1	ResearchX four-layer architecture on the Pi harness base. Solid arrows show the evidence flow; the double-headed links attach paper sources, compute and verification to their relevant layer. The human approval gate remains outside the Pi-owned runtime.	2
5.1	Illustrative composite pipeline assembled from the reviewed capabilities. . . .	11
7.1	Autonomous Kaggle experiment loop with ten-minute monitoring to a terminal kernel state.	14

List of Tables

3.1	Reviewed products: advertised capability and ResearchX comparison	8
3.2	Related systems: patterns adopted versus what is not taken	8
4.1	Discussed design versus proposed solution	10
5.1	Reviewed capability stages and the remaining ResearchX gap	11
6.1	Discovery and verification sources in the proposed design	12
7.1	Target ResearchX contract against reviewed capability classes	14

ABBREVIATIONS

API	Application Programming Interface
ARS	Academic Research Skills (stage-gate methodology reference)
CLI	Command-Line Interface
DOI	Digital Object Identifier
GPU	Graphics Processing Unit
OA	Open Access
Pi	Base agent harness runtime of ResearchX
S2	Semantic Scholar (current verifier dependency under review)
SOTA	State of the Art
TUI	Terminal User Interface

CHAPTER 1

INTRODUCTION

ResearchX is being developed as a terminal (TUI) research agent with a verifiable-output contract: a research question should produce source-backed evidence and a draft that is not final until a human approves it. The discussed architecture has four layers over a foundation. The Research layer searches, ranks and synthesises papers and audits code against paper claims. The Experiment layer is intended to isolate hypotheses, record runs and stop repeated failures. The Integrity layer is intended to verify citations, link empirical claims to logged results and enforce a human checkpoint. The Writing layer drafts from verified findings and applies a skeptic review pass. The foundation provides the agent orchestrator, tool manager, memory, prompt and context management, and logging with monitoring.

The intended data flow is Inputs to Research to Experiment to Integrity to Writing to Human Approval, with paper sources attached to Research, compute attached to Experiment, and verification sources attached to Integrity. Figure 1.1 redraws that contract on the actual runtime boundary.

The base claim of this review is that the base harness is the Pi agent harness [1]. Pi owns the interactive agent loop and the native TUI, model and provider routing, tool and extension registration with a session-start lifecycle, and sessions with researcher, reviewer, writer and verifier subagents. Related systems — a Feynman-class research agent [2], academic research skills [3] and DeepScientist [4] — are feature references only. None of them is the base runtime.

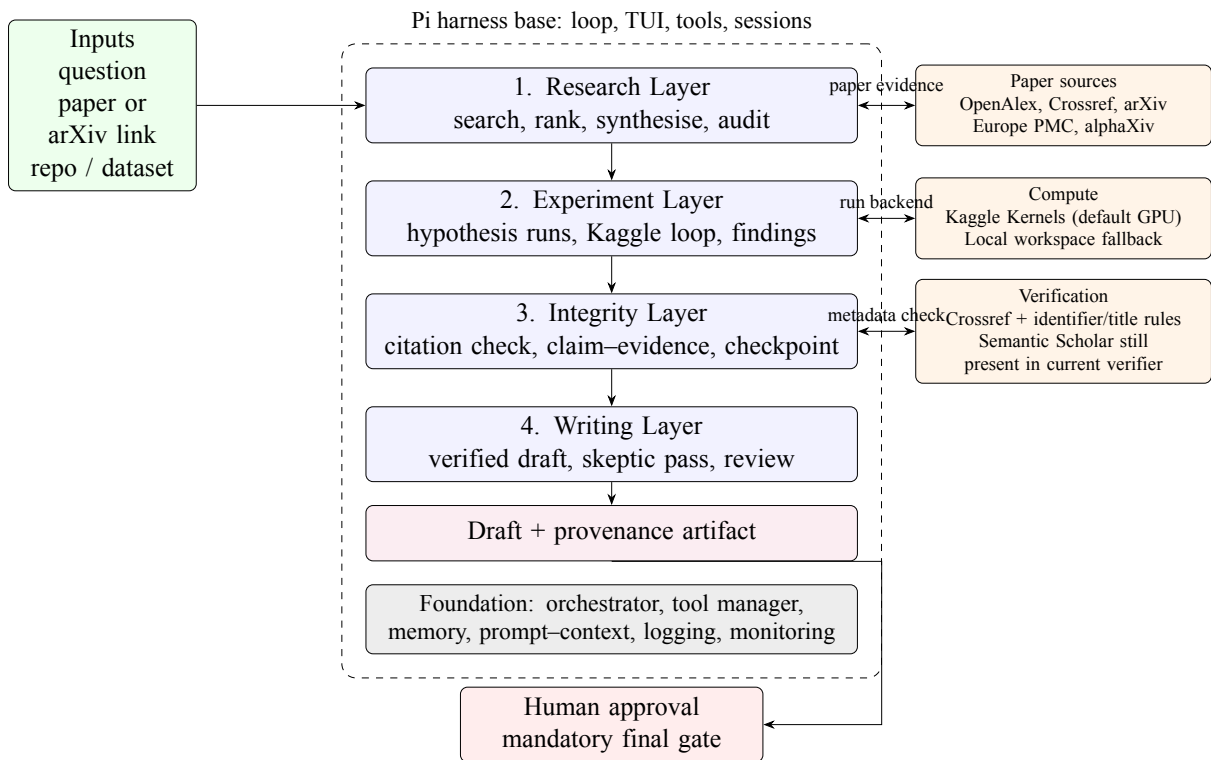


Figure 1.1: ResearchX four-layer architecture on the Pi harness base. Solid arrows show the evidence flow; the double-headed links attach paper sources, compute and verification to their relevant layer. The human approval gate remains outside the Pi-owned runtime.

CHAPTER 2

PROBLEM STATEMENT

2.1 Context

Research tooling creates a separation problem. Search systems return many papers but do not by themselves establish whether a paper supports the claim being written. Writing assistants can attach useful citations without proving that the cited metadata or interpretation is correct. Notebook and experiment systems can execute code while leaving source selection, claim tracing and review approval to the user. These are useful capabilities, but they do not automatically form one auditable research record.

2.2 Problem Formulation

Let a research question q and a corpus of candidate papers D be given. A research agent must produce a draft V such that every claim $c \in V$ maps to evidence $e(c)$, where $e(c)$ is either a verified citation or a logged experiment result:

$$V(D, q) = \{c_1, \dots, c_n\}, \quad \forall c_i \exists e(c_i) \in \text{Citations} \cup \text{Ledger}. \quad (2.1)$$

Existing systems break this contract at one of three points: they retrieve without verifying, they write without executing, or they execute without preserving the trail. The problem ResearchX solves is to close all three gaps inside one autonomous loop that a human finally approves.

2.3 Acceptance Criteria

The proposed system is evaluated against six acceptance criteria:

1. **Scholar-independent discovery.** Rank and cite from open sources with no load-bearing dependency on a rate-limited Scholar-class API.
2. **Executed experiments.** Every empirical claim traces to a kernel or workspace run recorded in an append-only ledger.
3. **Deterministic citation verdicts.** Each reference resolves to verified, mismatch, not-found or error by rule, not by model opinion.
4. **Preserved failures.** Dead ends, metrics and diagnoses persist so work is never repeated blindly and reviewers see the full trail.

5. **Bounded autonomy.** The agent polls, retries and reports on its own, but stuck runs are killed by guards and nothing ships without a human checkpoint.
6. **Open and replayable.** No paywall between the researcher and verification; every figure, table and claim replays from provenance records.

CHAPTER 3

LITERATURE REVIEW

3.1 Review of Related Theory

3.1.1 Agent Harness and Tool Extension Model

A research agent benefits from one clear interaction owner. In this design the harness owns the agent loop, streaming, slash commands, approvals, cancellation and event rendering, while domain capabilities arrive as first-class tools registered through an extension interface at session start. ResearchX follows this model: Pi owns the loop and the TUI, and ResearchX registers tools such as paper search and ranking, citation integrity, science databases, model endpoints, human checkpoint and Kaggle execution, together with workflow prompts (`lit`, `audit`, `draft`, `skeptic`, `replicate`, `recipe`, `kaggle`, `watch`) and skills for literature review, paper-code audit, replication and paper writing. A second prompt loop around the harness is therefore unnecessary and is deliberately avoided.

3.1.2 Paper Ranking with Citation Graphs

Ranking combines topical relevance with citation-graph and quality signals. For a candidate paper p , ResearchX scores a weighted combination of components:

$$\text{score}(p) = \sum_i w_i c_i(p) \quad (3.1)$$

with the current default weights $w = (0.3, 0.2, 0.2, 0.1, 0.1, 0.1)$ over topical relevance, citation impact, graph prestige, citation velocity, methodology quality and reproducibility. The graph features come from the OpenAlex Works API [9]: `cited_by_count`, `concepts` and `topics`, `open access locations`, `referenced` and `related works`, and `citation filters` with `batch-by-ID` and `citing-works expansion`. Preprint coverage comes from the arXiv API [11]. These are scoring-design weights, not a claim that the ranking has been empirically validated in this review.

3.1.3 Citation Verification

A cited reference is parsed as a DOI, an arXiv ID or a title, then checked for existence and metadata agreement. For title references, agreement uses the stricter of Jaccard word overlap and a character-level Levenshtein ratio:

$$\text{sim}(a, b) = \min(\text{Jaccard}(a, b), \text{LevRatio}(a, b)) \quad (3.2)$$

and a title reference is verified when the similarity passes 0.85 with year agreement within one year:

$$\text{verified} \iff \text{sim} \geq 0.85 \wedge |y_{\text{claimed}} - y_{\text{found}}| \leq 1 \quad (3.3)$$

For DOI and arXiv identifiers, the current implementation treats successful resolution plus year agreement as verification; title similarity is returned for inspection but is not the deciding condition. Each reference receives a deterministic verdict of verified, mismatch, not-found or error from the providers used by the current tool. The checked-out citation tool calls Crossref [10] and Semantic Scholar [15]; OpenAlex and arXiv are present in the database/search layer, but OpenAlex is not the current provider for this verifier. Consequently, the implementation is not yet Scholar-independent at the verification boundary.

3.1.4 Stage-Gate Integrity

Following skills-based methodology [3], every stage ends in an explicit gate: citations verified, experimental claims linked to ledger results, and a persisted `final_approval` checkpoint without which finalisation fails. Each literature run additionally ships a provenance record listing consulted versus accepted versus rejected sources, verification status and intermediate files, so a reviewer can replay the evidence chain.

3.2 Review of Related Works

3.2.1 Feynman-Class Research Agents

Feynman-class agents contribute the research workflow vocabulary: paper search with ranking, literature synthesis with source-per-claim tagging, code-versus-paper audit with evidence links, a typed research-run contract of papers, entities, tools, artifacts and verification states (`not_checked`, `inferred`, `partial`, `verified`, `blocked`, `failed`), citation-integrity checks, a human checkpoint, a skeptic and reviewer pass, and provenance sidecars [2]. ResearchX adopts these patterns as workflows and contracts on Pi. The Feynman runtime itself is not the base.

3.2.2 Execution and Guarding

Execution is the least complete part of the current design. ResearchX already has a Kaggle push/status/output path and Pi session branching, but the checked-out source does not yet evidence a dedicated hypothesis-branch ledger or a ResearchX-owned doom-loop detector. Those are design requirements, not results that this review can report as shipped. The correct next step is to persist each attempted hypothesis, failure reason, metric and stopping decision in a machine-readable run record before claiming experiment-level reproducibility.

3.2.3 Skills Methodology

Academic research skills contribute per-stage user confirmation and integrity gates between stages, including the single-pass skeptic review before delivery [3]. ResearchX re-implements those behaviours as independent contracts and checks instead of copying workflow text.

3.2.4 Local-First Research Studios

The DeepScientist repository (ResearAI/DeepScientist) [4] is a local-first research studio: one Git repository per research quest, visible progress, branch and worktree structure, a findings memory of what was tried with what failed and why plus metrics, preserved failed paths, a runner abstraction over external model CLIs, and human takeover of a stuck quest. Its entry points are a daemon with a Python TUI and a web workspace, and its public tool boundary is deliberately narrow. ResearchX adopts the durable-state thinking — quest-shaped outputs, run ledgers, preserved failures, human takeover — and maps it onto its outputs layout, findings ledger and provenance view, rather than adopting the daemon and runner control plane as a base.

3.2.5 Discovery Sources

OpenAlex [9] supplies the ranking graph. Crossref supplies DOI metadata [10]; arXiv provides fresh preprint coverage [11], Europe PMC with PubMed biomedical full text [12], alphaXiv trending-paper discovery [13], and Pi web search (auto, perplexity, exa, gemini routes) the broad web fallback. Domain entity grounding adds Ensembl, UniProt, ChEMBL, ClinVar, DepMap, GTEx and Open Targets.

3.2.6 Paid Research Platforms

The product pages reviewed show a more capable and more competitive baseline than the earlier draft assumed. Elicit [5] advertises paper search, research-agent reports, data extraction, tables, figures and sentence-level citations. Jenni AI [6] advertises source-linked writing, citation formatting and claim/review checks. Clusy.io [7] is an AI IDE for Jupyter notebooks that advertises runnable cloud experiments, branching and export. Scite [8] provides Smart Citations that classify later mentions as supporting, contrasting or merely discussing a work. These features are real strengths, so ResearchX cannot claim uniqueness from writing assistance, citation links, branching or experiment execution alone. The defensible comparison is narrower: this review asks whether the checked-out ResearchX implementation combines those capabilities with its own cross-source citation verdicts, local provenance artifacts and a mandatory human gate. Table 3.1 records that scoped comparison; it does not infer unadvertised limitations from a product page.

Table 3.1: Reviewed products: advertised capability and ResearchX comparison

Platform	Advertised strength	ResearchX comparison
Elicit	Reports, extraction, citations	Different source stack and gate
Jenni AI	Source-linked writing/review	Local artifacts and experiments
Clusy.io	Runnable, branched notebooks	Citation/provenance contract
Scite	Support/contrast citations	Local deterministic checks

Table 3.2: Related systems: patterns adopted versus what is not taken

System	Pattern adopted	Not taken
Feynman class	Ranking, synthesis, audit, skeptic pass	Runtime as base; Pi is the base
Execution controls	Kaggle push/status/output; Pi branching	Dedicated guards still partial
ARS methodology	Stage gates, skeptic review	Verbatim skill text
DeepScientist	Findings memory, preserved failures	Daemon/runner plane
Paid platforms	Reports, writing, notebooks, citation labels	Their pages do not establish ResearchX’s local checkpoint contract

3.3 Research Gap

This review identifies four engineering gaps in the sources examined. First, discovery, citation metadata and full-text services expose useful records but do not by themselves prove that a drafted interpretation is supported. Second, remote notebook execution needs an environment fingerprint and a durable run record before its result is reproducible. Third, tools that advertise strong literature or notebook workflows still differ in how they expose local artifacts, cross-source verdicts and human approval; this review does not assume those contracts are equivalent. Fourth, the current ResearchX source registry is broad, but this checkout includes no head-to-head evaluation of its many connectors against the OpenAlex/Crossref baseline. Those are measurable gaps, not evidence that other products cannot execute or verify.

CHAPTER 4

PREVIOUSLY USED ARCHITECTURE

4.1 The Discussed Design

The architecture the team first discussed is the four-layer pipeline of Figure 1.1: Inputs to Research to Experiment to Integrity to Writing to Human Approval, with external APIs, compute backends and verification sources connected to the relevant layers, and a foundation and orchestration bar underneath. The layer contracts were: paper search and ranking, literature synthesis, source-per-claim tagging and code-versus-paper auditing in Research; hypothesis-specific branches, free-tier and local execution backends, a findings ledger and doom-loop detection in Experiment; citation verification, claim-to-evidence traceability and a mandatory human approval checkpoint in Integrity; and verified-section drafting with a skeptic pass in Writing. The source list named arXiv, Semantic Scholar, OpenAlex and Crossref, with Colab, Kaggle Kernels and local Docker as compute options.

4.2 What Changed and Why

The proposed revision makes three corrections, and Table 4.1 records the result. First, the base is fixed as the Pi harness: Pi owns the loop, the TUI, tools and sessions, and ResearchX arrives as extensions rather than a second interaction runtime. Second, OpenAlex, Crossref, arXiv and Europe PMC form the intended discovery baseline, while the current citation verifier still calls Semantic Scholar; the report therefore treats Scholar-independence as unfinished at the verification boundary. Third, Kaggle Kernels are the default remote experiment path and the local workspace is the fallback, but the checkout still contains additional workbench provider code. Kaggle/local is the product policy for this design, not a claim that every other provider path has already been deleted.

Table 4.1: Discussed design versus proposed solution

Decision	Discussed design	Proposed design / audit status
Base runtime	Unfixed harness	Pi owns loop, TUI and tools; present in checkout
Source stack	Mixed API list	OpenAlex/Crossref/arXiv/Europe PMC baseline; verifier still calls Semantic Scholar
Compute	Colab/Kaggle/Docker list	Kaggle default, local fallback; extra provider code remains
Evidence	Approval checkpoint	Human checkpoint and provenance present; experiment ledger/guards remain partial

CHAPTER 5

COMPARATIVE ARCHITECTURE

5.1 A Composite Capability Pipeline

Stitching representative strengths from the reviewed systems gives the illustrative composite in Figure 5.1: claim mapping over open scholarly metadata, source-linked drafting, citation-context labels, a skeptic pass, durable experiment state and execution guards. This is a comparison model, not a benchmarked state-of-the-art result and not a claim that one product provides every stage.

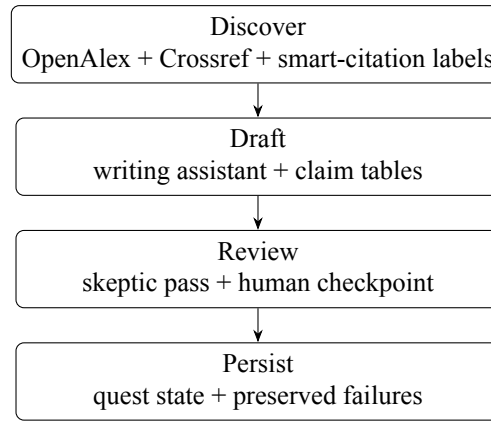


Figure 5.1: Illustrative composite pipeline assembled from the reviewed capabilities.

5.2 What the Composite Does Not Prove

Table 5.1 audits each stage against the six success criteria of Chapter 2. The table records what was and was not evidenced in the reviewed material; it is not a claim that the products have no additional integrations or capabilities. The key gap for ResearchX is integration: the target contract requires the stages to share identifiers, provenance and a human gate.

Table 5.1: Reviewed capability stages and the remaining ResearchX gap

Stage	Provides	Not evidenced here	ResearchX gap
Claim mapping	Tables and reports	Shared run ledger	Link to local provenance
Draft help	Source-linked writing	Deterministic checks	Preserve evidence IDs
Citation context	Support/contrast signals	Experiment linkage	Keep as context, not proof
Skeptic pass	Claim review	Human decision record	Connect to checkpoint
Quest state	Durable workspace	Cross-source verdicts	Join state to citations

CHAPTER 6

SOURCE STACK AND VERIFICATION LIMITS

The intended discovery baseline is independent of any single Scholar-class service. OpenAlex, Crossref, arXiv and Europe PMC/PubMed cover complementary metadata, graph, preprint and biomedical/full-text needs; alphaXiv and Pi web search provide additional discovery surfaces. Table 6.1 separates those discovery roles from the current citation-verification implementation.

Table 6.1: Discovery and verification sources in the proposed design

Function	Source or path	Role and limitation
Ranking + graph	OpenAlex Works API	Works, citations, topics and OA metadata
DOI record	Crossref REST API	Deposited DOI metadata; not claim truth
Preprints	arXiv API	Public e-print metadata and identifiers
Biomedical text	Europe PMC + PubMed	Search and OA full-text routes
Discovery	alphaXiv + Pi web search	Additional retrieval surfaces
Verdicts	Current Crossref + Semantic Scholar tool	Deterministic rules; not Scholar-independent

The current citation-integrity module still calls Semantic Scholar alongside Crossref. OpenAlex is available in the science-database layer but is not the provider used by that verifier. Therefore the report can claim an OpenAlex- centred discovery baseline, but it cannot claim Scholar-independent citation verification until the code path is removed or explicitly accepted as a dependency. The Europe PMC full-text route also applies to its Open Access subset, and arXiv use must follow the service’s API terms and acknowledgement request [11].

CHAPTER 7

PROPOSED SOLUTION

7.1 Design on the Pi Base

ResearchX keeps the four layers of Figure 1.1 and runs them on Pi: the harness owns the loop and the TUI while research workflows arrive as Pi extensions, tools, prompts and skills. In the checked-out source tree, the Research layer has PaperRank scoring, source search and code/audit workflows; the Integrity layer has citation-integrity and human-checkpoint tools; and the Writing layer has draft and skeptic workflows. The Kaggle tools implement the push, status and output path described in Figure 7.1. These are verified source-tree capabilities, not an independent evaluation of research quality.

The Experiment-layer contract remains partial at this audit date. The source tree does not yet evidence a dedicated hypothesis-branch findings ledger or a doom-loop detector, and the workbench still contains additional provider paths beyond the Kaggle/local default policy. The final report must therefore describe those items as planned integration work. Nothing in the proposed design requires a second interaction loop or a copied runtime.

Kaggle Kernels are the default experiment backend and the local workspace is the offline fallback. Authentication is through the Kaggle environment variables or the standard configuration file. Specifically, use `KAGGLE_USERNAME` and `KAGGLE_KEY`, or `~/.kaggle/kaggle.json`. The intended loop is autonomous and command-line driven, as shown in Figure 7.1: write the experiment locally under `outputs/experiments/<slug>/` with a single entry script plus helpers and a fast local smoke check only; push with GPU enabled unless the task is trivial and record the kernel slug and URL; poll the kernel status every ten minutes until a terminal state of complete, error, cancelled or failed, re-polling on running or queued; collect outputs into `outputs/kaggle/<slug>/`; and report the kernel URL, final state, key metrics from logs, artifact paths and a one-paragraph diagnosis on failure. Monitoring is delegated to a runner subagent so it survives context pressure, and a single blocking call covers push with poll with fetch. The planned guards are hypothesis isolation with one branch and slug per experiment, an append-only findings ledger of tried/failed-why/metrics, and a doom-loop detector that kills stuck runs and saves budget. The first of these is a design rule; the ledger and detector still require implementation and tests in the current checkout.

7.2 Why This Approach Is a Coherent Integration

The claim is not that ResearchX invents ranking, drafting help, notebook execution, citation context or skeptic review. Those capabilities already exist in different products. The proposed contribution is their integration around a shared artifact and approval contract: the agent can

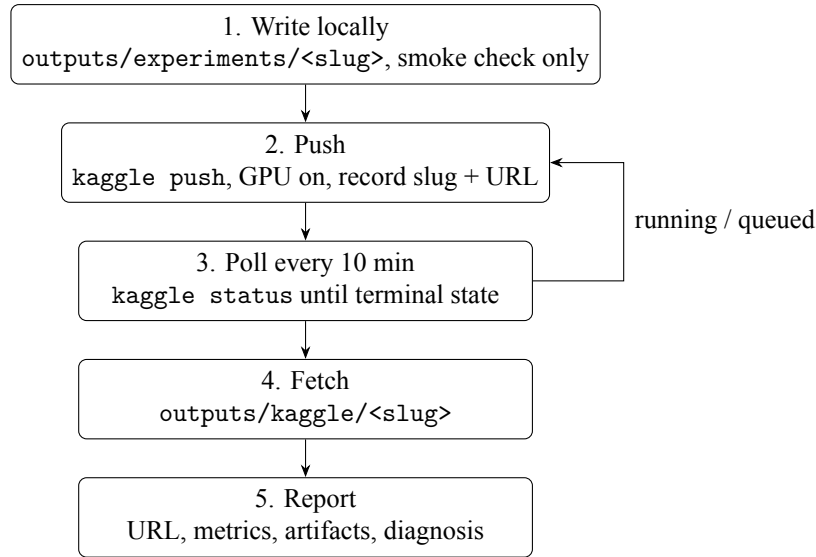


Figure 7.1: Autonomous Kaggle experiment loop with ten-minute monitoring to a terminal kernel state.

discover sources, write or push an experiment, collect outputs, attach provenance and route a draft through citation checks and a human gate. Table 7.1 compares that target contract with the capabilities evidenced in this review. It is a design claim, not a completed novelty or superiority result. The current checkout is not yet entitled to claim the full contract because the citation provider boundary and experiment ledger/guard work remain incomplete.

Table 7.1: Target ResearchX contract against reviewed capability classes

Approach	Strength	Missing from the reviewed evidence	ResearchX target
General chatbots	Flexible generation	Shared evidence record	Source-backed artifacts
Literature products	Search, reports, writing	Project-local gate/ledger varies	Integrate with provenance
Notebook agents	Runnable, branched experiments	Citation contract varies	Link runs to claims
Citation-context tools	Supporting or contrasting signals	Experiment linkage	Treat as context, not proof
ResearchX checkout	Pi workflows, Kaggle tools, checkpoint	Verifier/ledger guards partial	Complete and test the contract

CHAPTER 8

CONCLUSION AND NEXT WORK

ResearchX currently has a credible Pi-native foundation and several working research tools, but this audit does not support a claim of a finished, fully autonomous research system. The highest-priority work is to resolve the citation-provider policy, implement and test the hypothesis findings ledger and doom-loop guard, and decide whether the remaining workbench provider paths are in or out of scope. After that, pin environment fingerprints for remote runs and evaluate the broad connector registry against the OpenAlex/Crossref baseline. A human approval remains the final gate throughout.

All web sources below were accessed on 5 September 2026.

Bibliography

- [1] Earendil Works. Pi agent harness: pi-agent-core, pi-coding-agent, pi-ai and pi-tui. Agent loop, tool calling, state management, model routing and terminal UI. <https://github.com/earendil-works/pi>.
- [2] companion-inc. Feynman: research-first CLI agent with paper search, ranking, audit, synthesis and provenance workflows. <https://github.com/companion-inc/feynman>.
- [3] Imbad0202. Academic Research Skills: research, writing, review and stage-gate workflows. <https://github.com/imbad0202/academic-research-skills>.
- [4] ResearAI. DeepScientist: local-first research studio with quest state, research maps, branches and durable artifacts. <https://github.com/ResearAI/DeepScientist>.
- [5] Elicit. AI research agent, paper search, reports, data extraction and cited artifacts. <https://elicit.com>.
- [6] Jenni AI. AI academic writing, source-linked suggestions, citation formatting and review features. <https://jenni.ai>.
- [7] Clusy. AI IDE for Jupyter notebooks with agent-authored experiments, branching and cloud compute. <https://www.clusy.io>.
- [8] Scite. AI research platform and Smart Citations for supporting, contrasting and discussing evidence. <https://scite.ai>.
- [9] OpenAlex. Works API documentation: scholarly works, citations, topics, open access and query/filter support. <https://help.openalex.org/data/works/>.
- [10] Crossref. REST API documentation: public scholarly metadata retrieval, DOI records and query endpoints. <https://www.crossref.org/documentation/retrieve-metadata/rest-api/>.
- [11] arXiv. API access and interoperability guidance. <https://info.arxiv.org/help/api/index.html>.
- [12] Europe PMC. RESTful Web Service: search, citations, references, cross-references and Open Access full-text XML. <https://europepmc.org/RestfulWebService>.
- [13] alphaXiv. Paper discovery and analysis surface used by the ResearchX/Feynman workflow. <https://alphaxiv.org>.
- [14] Kaggle. Official Kaggle CLI: kernel push, status, run, download and output commands. <https://github.com/Kaggle/kaggle-cli>.
- [15] Semantic Scholar. Academic Graph API referenced by the current citation-integrity implementation. <https://api.semanticscholar.org/api-docs/>.